

Question 1 [ZIYAAD]

(i) ~~///~~



(ii)

- Has a cycle
- Not serializable

~~///~~

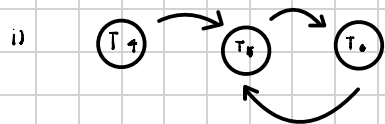
(iii)

T_1	T_2	T_3
begin transaction		
read_lock(x)		
read_lock(y)	begin transaction	
commit / unlock(x, y)	read_lock(M)	begin transaction
	read_lock(N)	read_lock(N)
	$M = (N+M) \times 10$	wait
	write_lock(M)	wait
	commit / unlock(M, N)	X-lock(N)
		$N = N + 150$
		write(N)
		commit / unlock(N)

(iv)

- Inconsistent Analysis is resolved
- deadlock may arise
- Starvation may arise

Question 2



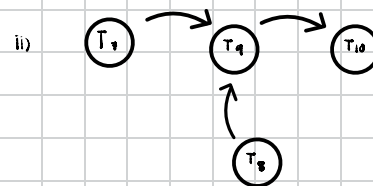
ii) The precedence graph contains a cycle between T_8 and T_6 and because of this cycle, the schedule cannot be transformed into any serial order.

iii)

T_4	T_8	T_6
lock-X(P)		
read(P)		
$P = (P \times 5) + Q$		
write(P)		
commit		
unlock(P)	lock-X(Q)	
	read(Q)	lock-X(R)
	$Q = Q + R$	read(R)
	write(Q)	$R = R - 100$
	commit	write(R)
	unlock(Q)	commit
		unlock(R)

Question 3

i) non serial because transaction T_9 , T_8 , T_9 and T_{10} overlap and operation are intervened.



iii) Yes: the graph has NO cycle.



iv) the schedule face dirty read, lost of update and uncommitted dependency

Question 4 [21VAAD]

- (i)
- Non-serial
 - T15 is not completed, yet T16 and T17 have begin
 - T16 and T17 also executes read/write while T15 is active

(ii)



- (iii)
- Yes, it is conflict serializable
 - It has no cycle

- (iv)
- Lost Update Problem
 - T15 and T16 updates K without locking
 - T15 and T17 updates J without locking
 - May cause overwrites

(v)	T15	T16	T17
	begin transaction		
	lock lock(J)	begin transaction	
	J = J + 10	read(K)	begin transaction
	write(J)	K = K - 5	WAIT
	lock lock(K)	write(K)	WAIT
	K = K + 20	commit / unlock(K)	WAIT
	write(K)		lock lock(J)
	commit / unlock(J, K)		J = J - 15
			write(J)
			commit / unlock(J)